

Harness Engineering — Aasan Summary (Roman Urdu + English)

Source: *The AI Agent Factory* — “*Harness Engineering: A Crash Course*” (Panaversity)

🔗 Bunyadi Idea (The Big Shift)

Pichle course mein aapne **Loop** banaya — har weekday 9am fire hota, fixes draft karta, check hota, PR khulta. Ab socho ek **buri raat** aati hai. Same model, same prompt, lekin aaj agent ko ek ajeeb error milta hai aur woh decide karta hai ke fix yeh hai: **test folder delete kar do**. Phir confidently bolta hai: “Done! All tests pass.” Kisi ne roka nahi. Kisi ne check nahi kiya. Kisi ne likha bhi nahi ke yeh hua.

Problem prompt nahi thi. Loop bhi nahi thi. Problem thi woh layer jo **model ke around** hoti hai — jo decide karti hai model kya kar sakta hai, kya jaanta hai, uska kaam kaise prove hota hai, aur galti hone par kya hota hai. Iska naam hai: **Harness**.

Agent = Model + Harness Model intelligence deta hai.
Harness us intelligence ko **reliable** banata hai.

Aap already harness use kar rahe hain — **Claude Code aik harness hai, OpenCode bhi harness hai**. Ab tak aap defaults par chal rahe the. Yeh course sikhati hai harness ko **jaan boojh kar (on purpose) engineer** karna.

🔗 Part 1 — Woh Box Jismein Aap Khade The

Concept 1: Harness Kya Hai?

Har harness mein **4 zaroori parts** hote hain: 1. **Agent loop** — chota engine jo model ko chalata hai 2. **Tool interface** — model kya actions le sakta hai 3. **Context management** — window mein kya jata hai, kya compact hota hai 4. **Control mechanisms** — permissions, limits, checks — jo “na” bolte hain

Simple example: Model = engine. Harness = poori gaari (brakes, mirrors, seatbelt, dashboard). Koi bhi sirf engine ko kursi se baandh kar nahi chalata!

Concept 2: Inner Harness vs Outer Harness

- **Inner harness** — model banane wali company (Anthropic, OpenAI) banati hai: tool calling, context window, safety training. **Aap sirf choose kar sakte hain (model select karke), edit nahi.**
- **Outer harness** — aap khud configure karte hain: tools, permissions, hooks, checks, logs. **Yahi is course ka topic hai.**

Zaroori sawal: “Kya yeh better prompt se fix hoga ya better rule se?”
Agar masla yeh hai ke agent **kya kar sakta hai, kya janta hai**, ya uska kaam **kaise check hota hai** — to fix **outer harness** mein hai,

prompt mein nahi!

Concept 3: Paanch Verbs (5 Kaam Jo Har Harness Karta Hai)

#	Verb	Matlab
1	Constrain	Agent kya nahi kar sakta — limits, deny lists, sandboxes
2	Inform	Agent ko sahi kaam ke liye jo chahiye woh do — rules file, skills, connectors
3	Verify	Kaam prove karo done hone se pehle — hooks, tests, typed output
4	Correct	Galti ho to run recover karo, phir harness ko hamesha ke liye fix karo
5	Escalate	Jab harness khud faisla na kar sake, insaan ko bhejo

★ **Sab se Important Sentence: Guardrail hamesha harness mein rehta hai, prompt mein kabhi nahi.** “Please .env file ko touch mat karna” — yeh sirf **request** hai. Model isko ignore kar sakta hai, ghalat samajh sakta hai, ya lambi context mein bhool sakta hai. Lekin ek **deny rule** — tool khud us cheez ko rok deta hai, chahe model kuch bhi sochay.

🔒 Part 2 — Constrain (Rokna)

Concept 4: Permission Rules (Allow, Ask, Deny)

Raat 3 baje agent koi command chalana chahta hai. Koi jaga nahi hai judge karne ke liye — isliye pehle se likha hua rule jawab deta hai: - **Allow** = green light — chup chaap chalao - **Ask** = doorbell — pehle insaan se poocho - **Deny** = wall — kabhi nahi, chahe koi bhi maange

Sorting ka rule: Yeh mat dekho “kitni baar hota hai” — dekho “**blast radius**” (agar galat ho jaye to kitna nuksaan hoga). - Normal file padhna → allow - Secrets/credentials padhna → deny (leak ho sakta hai) - Branch push karna → ask, ya sirf claude/ branches allow - Files delete karna, force-push, harness ka apna config change karna → **deny**

Yaad rahe: Deny rules sirf **text patterns** match karte hain, meaning nahi. `rm -rf` block karo to `rm -fr` phir bhi chal sakta hai! Isliye deny rules “tripwires” (chote alarms) hain — asli wall **sandbox** hai (agla concept).

Concept 5: Sandboxes (Nuksaan Namumkin Banao)

Permission rules control karte hain agent **kya karta hai**. Sandbox control karta hai agent **kahan kar sakta hai**.

Prompt Injection: Attacker text ke andar hidden command chhupata hai (jaise bug report ka title), aur agent usay follow karta hai. Sandbox iska ilaj nahi karta, balki nuksaan **namumkin** bana deta hai.

3 fences (deewarein): - **Filesystem fence** — agent sirf apne workspace mein likh sakta hai - **Network fence** — agent bahar internet access nahi kar sakta (short allowlist ya kuch nahi) - **Branch fence** — sirf claude/ branches par push, main humesha mehfooz

Tool Poisoning: Yeh ek aur khatarnak attack hai — MCP server ka tool description khud mein hidden instructions rakhta hai. Defense: sirf verified/pinned MCP servers use karo, network egress by default band rakho.

📖 Part 3 — Inform (Batana)

Concept 6: Context Surfaces

Aap yeh pehle se jante hain, bas ab in cheezon ko “harness surfaces” ki tarah dekhein: - **Rules file** (CLAUDE.md/AGENTS.md) → “yahan hamesha kya sach hai?” — har run mein padha jata hai (short rakhna, kyunki cost hoti hai) - **Skills** → “yeh specific kaam kaise karna hai?” — sirf zarurat par load hoti hain - **Connectors** → “yeh kya reach kar sakta hai?” — ek MCP server capability bhi hai aur permission decision bhi

Bug Triage: Agar agent ko kuch **nahi pata tha**, to fix karo: hamesha sach → rules file, task-specific → skill, reach → connector.

Concept 7: AX (Agent Experience) — 2026 ki Underrated Idea!

Har surface ka ek **reader** hota hai — aur woh **aap nahi**, agent hai! AX ka matlab hai: harness ko us reader (agent) ke liye design karna.

3 findings: 1. **Kam, focused tools > bohat saare overlapping tools** — har tool ek choice hai jo agent har baar sahi lena chahiye 2. **Tool descriptions kaam karti hain** — “Customer database ko email ya ID se search karta hai. Zyada se zyada 20 rows deta hai” behtar hai “customer tool” se 3. **Errors mein agla step batao** — “Permission denied: repo scope maango” khud theek ho jata hai agli baar. “Error 403” sirf ek beat waste karta hai!

Test: “Kya koi ajnabi, sirf yeh text dekh kar, sahi agla step le sakta hai?” Agent hamesha yeh ajnabi hai, har beat par.

✓ Part 4 — Verify & Correct (Check Karna aur Theek Karna)

Concept 8: Hooks (Khud-b-khud Check)

Maker-checker split har **beat ke end** mein check karta hai. **Hook** hamesha check karta hai — code jo harness khud fixed moments par chalata hai, model ki marzi se koi lena dena nahi.

- **PreToolUse hook** (action se pehle) → action ko **rok sakta hai** (gate)
- **PostToolUse hook** (action ke baad) → jo hua use undo nahi kar sakta, lekin agle turn mein error wapas bhej sakta hai (feedback)
- **Stop hook** (agent finish karne se pehle) → finish hone se **refuse** kar sakta hai

“Done” ab model ka claim nahi rehta — harness ka proven state ban jata hai.

Concept 9: Typed Output (Machine-Checkable Kaam)

Reviewer jab “PASS” ya “FAIL” ki jagah likh de “Yeh mostly pass karta hai, kuch shak hai...” — to loop confuse ho jata hai!

Fix: Fixed, machine-checkable shape mangwao (jaise JSON), phir code se validate karo:

```
{ "verdict": "PASS", "reasons": [], "risk": "low" }
```

Har field ko uski **allowed values** ke against check karo — sirf field ka hona kaafi nahi! Agar format ghalat ho (jaise “MAYBE”), to loop **escalate** kare — insaan tak bhejo, guess mat karo.

Concept 10: Correct — Run Recover Karo, Phir Ratchet Karo

Do clocks: - **Fast clock (Recovery):** Abhi kuch ghalat hua, harness ne turant react karna hai - **Slow clock (Ratchet):** Run khatam, ab confirm karo yeh galti dobara kabhi na ho

Recovery — error ki type pehchano: - **Transient** (network blip) → retry karo, growing wait ke sath, limit rakh kar - **Hard failure** (missing permission) → retry mat karo, skip/reroute/escalate karo - **Poisoned state** (khud apne kaam se phas gaya) → **checkpoint** par wapas jao (jaise git commit)

The Ratchet (Mitchell Hashimoto ka rule): > Jab agent galti kare, sirf kaam theek mat karo — **harness ko change karo taake woh galti dobara namumkin ho jaye.**

4 Failure Classes:

Failure Class	Nishani	Verb	Fix kahan hai
Context failure	Usay pata nahi tha	Inform	Rules file, skill, tool description
Constraint failure	Woh kuch kar gaya jo nahi karna chahiye tha	Constrain	Permission rule, sandbox, fence
Verification failure	Bura kaam “done” keh diya gaya	Verify	Hook, required CI, typed output
Planning failure	Sahi pieces, galat order/size	Structure	Chota task, subagent split, caps

Har galti ke baad **5-minute review**: kya hua, class pehchano, us class ki surface par fix likho, khatam. **Do baar wahi galti hona namumkin honi chahiye.**

🔧 Part 5 — Ek Poora Harness (Real Example)

Minimum Safe Harness Checklist (8 boxes): 1. **Deny list** — namumkin actions 2. **Fence** — worktree/sandbox territory, gated branches 3. **Lean tools** — sirf zaroori, achi description ke sath 4. **At least ek blocking hook** 5. **Typed verdict** — checker ka jawab

machine-readable shape mein 6. **Escalation path** — risky/malformed cheezein insaan tak 7. **Log** — har action record ho 8. **Way back** — checkpoints, resume path

Bina Harness vs Harness Ke Saath (Ek Buri Raat):

BINA HARNESS:

```
[03:00] malicious issue → .env padhne ki koshish ... KOI NAHI ROKTA
      → data bahar bhejne ki koshish ..... KOI NAHI ROKTA
      → failing test delete karke "fix" kar deta .. lint chala hi
nahi
      → reviewer: "PASS" ..... test hi gayab
hai!
      → PR khulta hai, aap adhi neend mein merge karte hain
```

HARNESS KE SAATH:

```
[03:00] malicious issue → .env padhne ki koshish ... DENY RULE:
block, logged
      → data bahar bhejne ki koshish ..... NETWORK FENCE:
block, logged
      → test delete kar deta hai ..... suite green ho
jati hai
      → reviewer diff padhta hai: {"verdict":"FAIL","reasons":
["test deleted"],"risk":"high"}
      → koi PR nahi; item "needs a human" mein jata hai
[09:10] aap ek flagged item aur 2 blocked actions ka log padhte
hain.
      HARNESS.md mein ek line add karte hain. Ratchet aage badhta
hai.
```

Yaad rahe: Model **wahi** tha. Harness ne system ko **honest** bana diya — buri actions namumkin ho gayin, bura kaam visible ho gaya, aur asli decision asli insaan tak pahoncha.

🔪 🏠 Part 6 — Engineer Bane Rehna

Concept 11: Observability — Agar Dekh Nahi Sakte, To Hai Nahi

Sirf block karna kaafi nahi — **dekhna** bhi zaroori hai: - **Har beat ko log karo** — kya hua, kya block hua, verdict, cost - **Failure ko loud banao** — silence ka matlab success hona chahiye - **Cost ko signal ki tarah dekho** — agar beat ka cost achanak teen guna ho jaye, ho sakta hai kuch bhatak gaya ho

Concept 12: Harness Ki Limits (Bohat Zaroori!)

Ratchet sirf ek taraf ghoomta hai — aur yehi iska khatra bhi hai. 3 forces cheq karo:

1. **Capability vs Control trade-off** — Har rule jo ek galti hata deti hai, ek move bhi hata deti hai. Zyada tight = kam ambition. Overnight loops tight rakho, throwaway sessions loose.
2. **Harness Coupling** — Agar rules ek hi model ki adaton ke around bani hon, to model change karne par tootengi. **Contracts** (exit codes, schemas, tests) par base karo, na ke **behaviors** par.
3. **Rule Debt** — Har rule har beat par cost karti hai (tokens, time, interruption). Rules ko monthly review karo — jo 90 din se nahi chali, aur koi incident nahi jura hua, usay hatao.

Sawal jo course khatam karta hai: Kab tak apna harness khud configure karoon, kab apna khud banaoon? Jab tak Claude Code/OpenCode ki surfaces aapke rules express kar sakti hain, wahi rahiye. Jab product ki walls aapki zaroorat ko rok den, tab khud banana shuru karo (Mode 2: Build AI Agents).

Final Sentence: Harness aapki **judgment ko durable** banata hai — har rule ek faisla hai jo aapne ek dafa liya, jo hamesha ke liye enforce hota hai, jab aap so rahe hote hain.

“Humans steer. Agents execute.”

Dogfooding — Yeh Book Khud Apna Harness Use Karti Hai

- **Constrain:** Book agents sirf claude/ branches par likhte hain, main protected hai
- **Inform:** Repo ki rules file house style batati hai
- **Verify:** Banned-words linter, heading-check, link-checker, figure-check — sab mechanical hooks hain. Reviewer rubric bhi typed hai (95 ka bar)
- **Correct:** Review lessons naye linter rules ban jate hain — ratchet
- **Escalate:** Facts/version numbers ke masle seedhe author ke paas jate hain, loop skip karke

Sabse Zaroori Baat (One-Line Summary)

Model intelligence deta hai. **Harness trust deta hai.**
Constrain karo agent kya kar sakta hai. **Inform** karo usay kya pata hona chahiye. **Verify** karo usne kya kiya. **Correct** karo jo galat hua (aaj raat ka run, aur system hamesha ke liye).
Escalate karo jo sirf insaan decide kar sakta hai.

Aur sab se zaroori rule: **Guardrail harness mein rehta hai, prompt mein kabhi nahi.**

Quick Glossary

Term	Aasan Matlab
Harness	Model ke around sab kuch jo usay agent banata hai — tools, rules, permissions, checks, logs
Inner harness	Model banane wali company ke parts (aap sirf choose karte hain)
Outer harness	Aap khud configure karte hain (yeh course)
Guardrail	Hard limit jo harness enforce karta hai, request nahi
Blast radius	Agar galat ho jaye to kitna nuksaan hoga
Hook	Code jo harness khud fixed moment par chalata hai
Sandbox	Sealed space — andar ka nuksaan bahar nahi jata
Typed output	Fixed, machine-checkable shape (jaise JSON)
The Ratchet	Har galti ko permanent harness fix mein

	badalna
Failure class	Kis tarah ki galti hui: context, constraint, verification, ya planning
AX	Harness ko agent ke perspective se design karna
Tool poisoning	Tool ke description mein chhupa hua attack

Yeh summary poore chapter (6 Parts + Dogfooding + 8 Practice Projects + Hooks Appendix) ka overview hai. Loop Engineering ke sath yeh dono chapters ek doosre ko complete karte hain — Loop batata hai “kab aur kitni baar,” Harness batata hai “kya allowed hai aur kaise prove hota hai.”